Università di Genova

UNIVERSITÀ DEGLI STUDI DI GENOVA

DIBRIS

DEPARTMENT OF INFORMATICS,
BIOENGINEERING, ROBOTICS AND SYSTEM ENGINEERING

MODELLING AND CONTROL OF MANIPULATORS

Third Assignment

Jacobian Matrices and Inverse Kinematics

Author:

HUSSEIN MOHAMED ABDELAZIM
MOHAMED MOSTAFA ELNAGGAR
Student ID: S8786438

AHMED MOHAMED AHMED
MOHAMED TAWAKOL
Student ID: S7952397

Professors:

Enrico Simetti
Giorgio Cannata

Tutors:

Luca Tarasi
Simone Borelli

December 15, 2025

Contents

1 Assignment description **3**
1.1 Exercise 1 . . . 3
1.2 Exercise 2 . . . 3

2 Exercise 1 **5**
2.1 Step 1: Define the Tool Frame . . . 5
2.2 Step 2: Initialize Geometric and Kinematic Models . . . 5
2.3 Step 3: Compute Transformation from Tool to Base . . . 5
2.4 Results . . . 5

3 Exercise 2 **6**
3.1 Goal Definition . . . 6
3.2 Control Algorithm . . . 6
3.3 Results . . . 6
3.3.1 Initial Configuration . . . 6
3.3.2 Final Configuration . . . 7
3.3.3 Graphical Results . . . 7

Mathematical expression	Definition	MATLAB expression
$< w >$	World Coordinate Frame	w
${}^{a}_{b}R$	Rotation matrix of frame $< b >$ with respect to frame $< a >$	aRb
${}^{a}_{b}T$	Transformation matrix of frame $< b >$ with respect to frame $< a >$	aTb
${}^{a}O_{b}$	Vector defining frame $< b >$ wit respect to frame $< a >$	aOb

Table 1: Nomenclature Table

1 Assignment description

The third assignment of Modelling and Control of Manipulators focuses on Inverse Kinematics (IK) control of a robotic manipulator.

The third assignment consists of three exercises. You are asked to:

- Download the .zip file called from the Aulaweb page of this course.
- Implement the code to solve the exercises on MATLAB by filling in the predefined files.
- Write a report motivating your answers, following the predefined format on this document.
- **Do NOT put your code in the report!**

1.1 Exercise 1

Given the geometric model of an industrial manipulator in Figure 1, you have to add a tool frame. The tool frame is rigidly attached to the robot end-effector according to the following specifications: $\Gamma_{t/e} = [\pi/10, 0, \pi/6], {}^{e}O_t = [0.3, 0.1, 0]^\top (m)$ where $\Gamma_{t/e}$ represents the YPR values from end effector frame to tool frame.

To complete this task you should modify the class *geometricModel* by adding a new method called *getToolTransformWrtBase*.

1.2 Exercise 2

Implement an inverse kinematic control loop to control the tool of the manipulator. You should be able to complete this exercise by using the MATLAB classes implemented for the previous assignment (*geometricModel*,*kinematicModel*), and also you need to implement a new class *cartesianControl* (see the template attached). The initial joint position is $q = [\pi/2, -\pi/4, 0, -\pi/4, 0, 0.15, \pi/4]^\top$, expressed in radians and meters. The procedure can be split into the following phases

Q2.1 Compute the cartesian error between the robot end-effector frame ${}^{b}_{t}T$ and the goal frame ${}^{b}_{g}T$.

The goal frame must be defined knowing that:

- The goal position with respect to the base frame is ${}^{b}O_g = [0.2, -0.7, 0.3]^\top$ (m)
- The goal frame is rotated by the YPR angles $\Gamma_g = [0, 1.57, 0]^\top$ (rad) with respect to the base frame.

Q2.2 Compute the desired angular and linear reference velocities of the tool with respect to the base: ${}^{b}\nu^{*}_{t/b} = \begin{bmatrix} \kappa_a & 0 \\ 0 & \kappa_l \end{bmatrix} \cdot {}^{b}e$, such that $\kappa_a = 0.8$,$\kappa_l = 0.8$ is the gain.

Q2.3 Compute the desired joint velocities $\dot{\tilde{q}}$

Q2.4 Simulate the robot motion by implementing the function: *"KinematicSimulation()"* for integrating the joint velocities in time.

Q2.5 Compute the end effector and tool linear and angular velocities with respect to the base frame projected on the base frame.

Remark 1: All the methods must be implemented for a generic serial manipulator. For instance, joint types, and the number of joints should be parameters.

Remark 2: The class *plotManipulators* is provided for plot generation. You must not modify its implementation, or its calls in *main.m.*

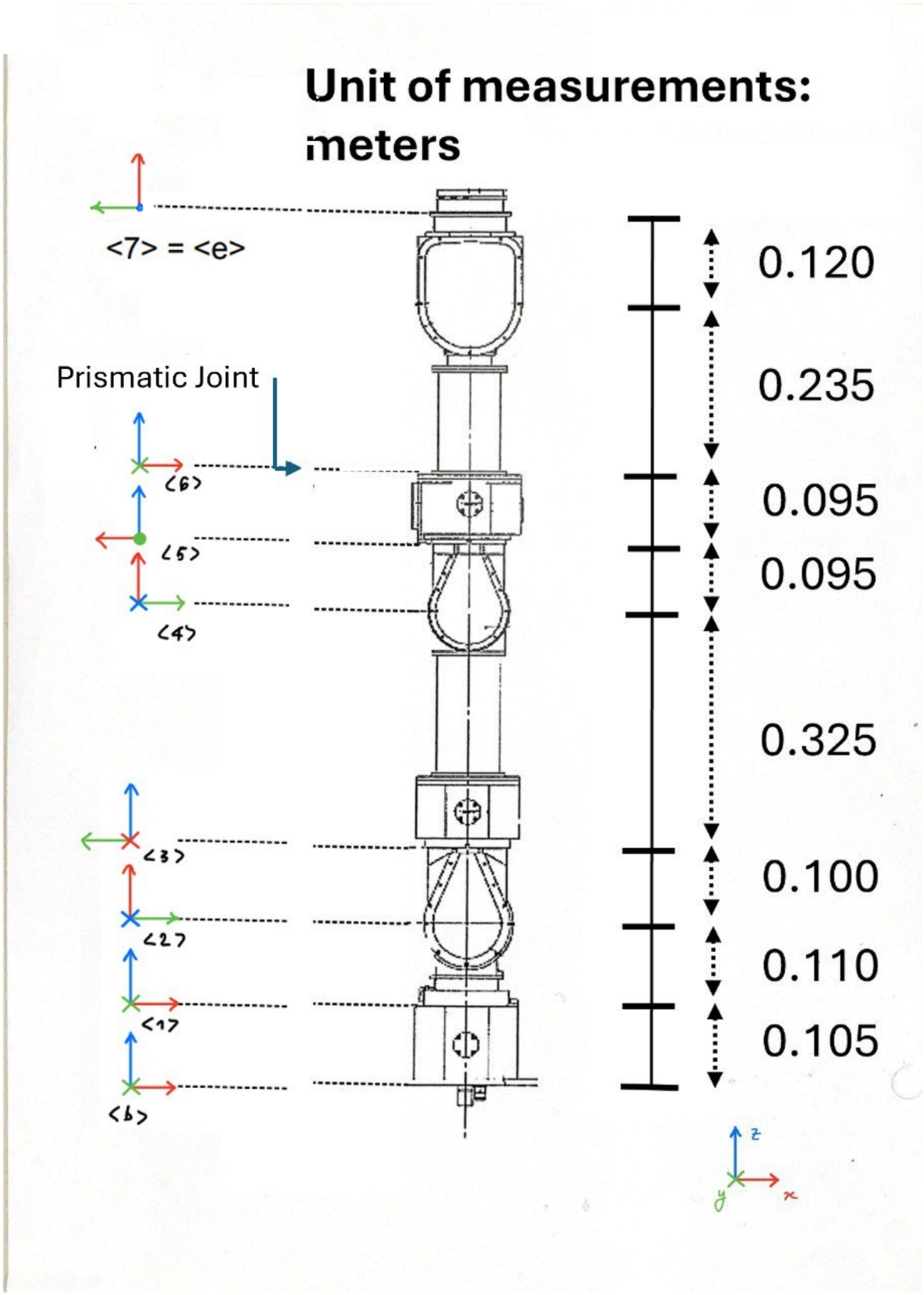


Figure 1: Manipulator CAD

2 Exercise 1

In this exercise, we are tasked with extending the geometric model of an industrial manipulator by defining a tool frame rigidly attached to the end-effector. This transformation allows for precise control of the tool center point (TCP).

The specifications for the tool frame are provided as follows:

- Orientation (YPR) relative to the end-effector: $\Gamma_{t/e} = [\pi/10, 0, \pi/6]^T$ rad.
- Position relative to the end-effector: ${}^eO_t = [0.3, 0.1, 0.0]^T$ m.

2.1 Step 1: Define the Tool Frame

The tool frame is defined relative to the end-effector frame. We compute the rotation matrix ${}^eR_t$ using the `YPRToRot` function with the given angles. Combined with the translation vector ${}^eO_t$, the homogeneous transformation matrix ${}^eT_t$ is constructed:

$$ {}^eT_t = \begin{bmatrix} {}^eR_t & {}^eO_t \\ \mathbf{0}^T & 1 \end{bmatrix} \tag{1} $$

Using the values from our simulation output, the resulting matrix is:

$$ {}^eT_t = \begin{bmatrix} 0.9511 & -0.2676 & 0.1545 & 0.3000 \\ 0.3090 & 0.8236 & -0.4755 & 0.1000 \\ 0 & 0.5000 & 0.8660 & 0 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix} \tag{2} $$

2.2 Step 2: Initialize Geometric and Kinematic Models

We initialized the geometric model (GM) with the provided robot structure and the computed tool frame. The kinematic model (KM) was then instantiated based on this geometric model to facilitate Jacobian computations.

2.3 Step 3: Compute Transformation from Tool to Base

To obtain the tool's pose in the world frame, we implemented the method `getToolTransformWrtBase`. This method computes the chain of transformations:

$$ {}^bT_t(q) = {}^b\,T_e(q) \cdot {}^e\,T_t \tag{3} $$

where ${}^bT_e(q)$ is the forward kinematics of the end-effector relative to the base.

2.4 Results

Evaluating the direct geometry at the initial configuration $q_{init} = [\pi/2, -\pi/4, 0, -\pi/4, 0, 0.15, \pi/4]^T$, the computed transformation matrix of the tool with respect to the base is:

$$ {}^bT_t(q_{init}) = \begin{bmatrix} 0.0000 & 0.5000 & 0.8660 & 0.0000 \\ -0.4540 & 0.7716 & -0.4455 & -1.1369 \\ -0.8910 & -0.3932 & 0.2270 & 0.2327 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix} \tag{4} $$

This matrix represents the starting pose for the trajectory tracking task in Exercise 2.

3 Exercise 2

In this exercise, we implement an inverse kinematic control loop to control the manipulator's tool. The goal is to minimize the cartesian error between the current tool pose and a desired goal pose using a Jacobian-based control scheme.

3.1 Goal Definition

The goal frame is defined by a target position ${}^bP_g$ and orientation ${}^b\Gamma_g$. **Note:** Following the assignment guidelines for convergence issues (Plan B), we utilized the alternative target position $y = -0.8$ m.

The goal specifications are:

- Target Position: ${}^bP_g = [0.2, -0.8, 0.3]^T$ m.
- Target Orientation: ${}^b\Gamma_g = [0, 1.57, 0]^T$ rad.

The resulting goal transformation matrix ${}^bT_g$ is:

$$ {}^bT_g = \begin{bmatrix} 0.0008 & 0 & 1.0000 & 0.2000 \\ 0 & 1.0000 & 0 & -0.8000 \\ -1.0000 & 0 & 0.0008 & 0.3000 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix} \tag{5} $$

3.2 Control Algorithm

The control loop performs the following steps at each iteration:

1. **Compute Cartesian Error:**
   - Linear error: $e_p = {}^bP_g - {}^bP_t$
   - Rotational error: e_o derived from the axis-angle representation of ${}^tR_g = {}^bR_t^T \, {}^bR_g$.
2. **Reference Velocities:** $\dot{x}^* = [k_a e_o^T, k_l e_p^T]^T$ with gains $k_a = k_l = 0.8$.
3. **Inverse Kinematics:** $\dot{q} = J^\dagger \dot{x}^*$ using the pseudoinverse.
4. **Integration:** $q_{k+1} = q_k + \dot{q}\Delta t$, respecting joint limits.

3.3 Results

The simulation was run for 15 seconds. Below are the detailed numerical results for the initial and final configurations, demonstrating convergence.

3.3.1 Initial Configuration

At $t = 0$, the errors and velocities were:

Position Error (e_p) and **Orientation Error (e_o)**:

$$ e_p = \begin{bmatrix} 0.2000 \\ 0.3369 \\ 0.0673 \end{bmatrix}, \quad e_o = \begin{bmatrix} 0.4601 \\ 0.1226 \\ 0.5140 \end{bmatrix} \tag{6} $$

Desired Cartesian Velocities ($\dot{x}$):

$$ \dot{x} = \begin{bmatrix} 0.3681 & 0.0981 & 0.4112 & 0.1600 & 0.2696 & 0.0539 \end{bmatrix}^T \tag{7} $$

Joint Velocities ($\dot{q}$):

$$ \dot{q} = \begin{bmatrix} -0.5197 & 0.1481 & 1.3164 & -0.0598 & -1.0289 & -0.1209 & 0.4565 \end{bmatrix}^T \tag{8} $$

Error Norms:

- $||e_p||_{init} = 0.3976$
- $||e_o||_{init} = 0.7007$

3.3.2 Final Configuration

At the end of the simulation ($t = 15$s), the robot converged to the target:

Position Error (e_p) and **Orientation Error (e_o)**:

$$e_p = \begin{bmatrix} 0.0036 \\ 0.0059 \\ 0.0011 \end{bmatrix}, \quad e_o = \begin{bmatrix} 0.0083 \\ 0.0021 \\ 0.0090 \end{bmatrix} \tag{9}$$

Desired Cartesian Velocities ($\dot{x}$):

$$\dot{x} = \begin{bmatrix} 0.0066 & 0.0017 & 0.0072 & 0.0028 & 0.0047 & 0.0009 \end{bmatrix}^T \tag{10}$$

Joint Velocities ($\dot{q}$):

$$\dot{q} = \begin{bmatrix} 0.0002 & -0.0005 & 0.0077 & -0.0026 & -0.0016 & -0.0006 & 0.0083 \end{bmatrix}^T \tag{11}$$

Error Norms:

- $||e_p||_{final} = 0.0070$
- $||e_o||_{final} = 0.0124$

These values confirm that the errors have dropped significantly below the required threshold, ensuring precise positioning.

3.3.3 Graphical Results

The following figures illustrate the robot's motion and velocity profiles.

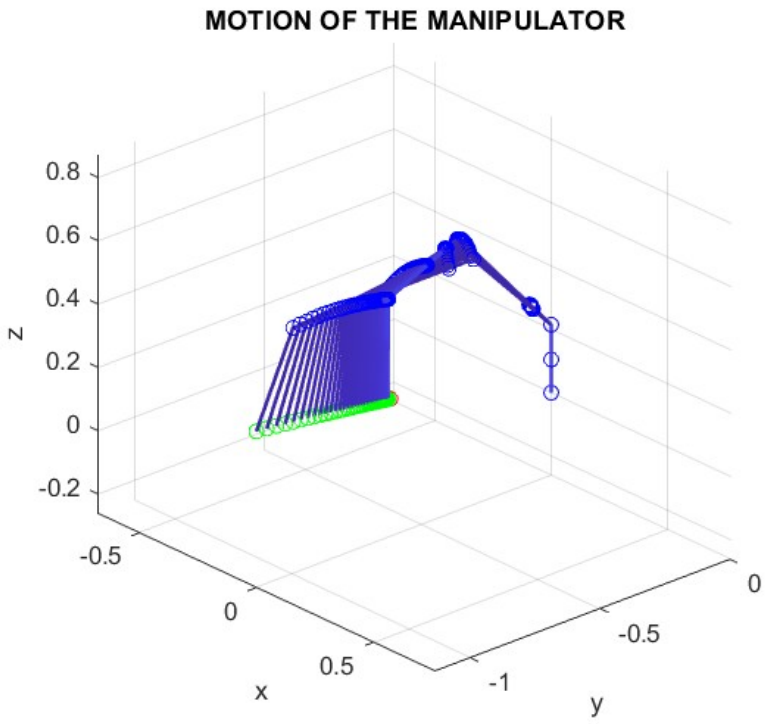


Figure 2: Stroboscopic motion of the manipulator towards the goal.

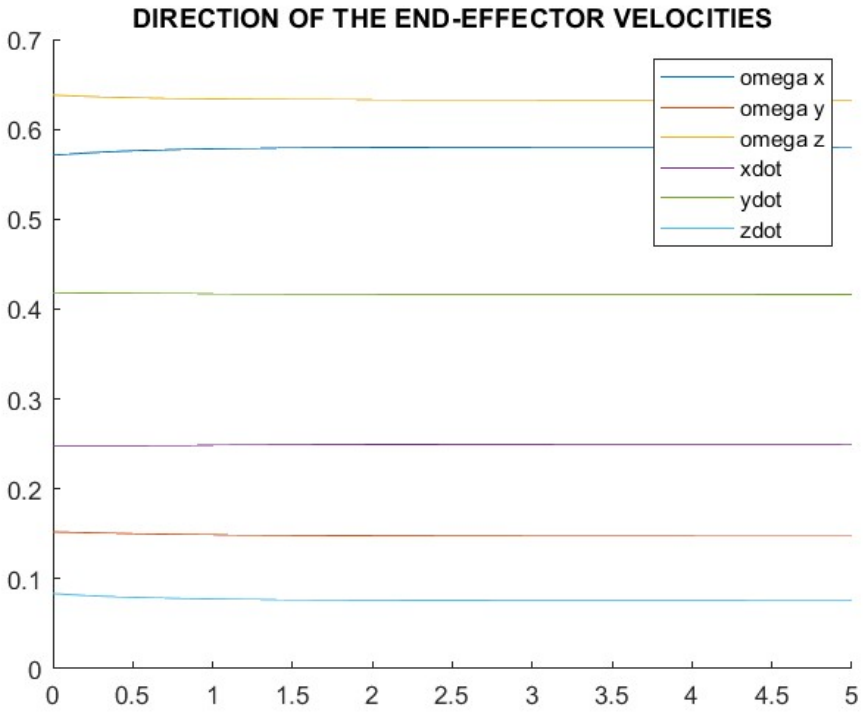


Figure 3: Cartesian reference velocities converging to zero.

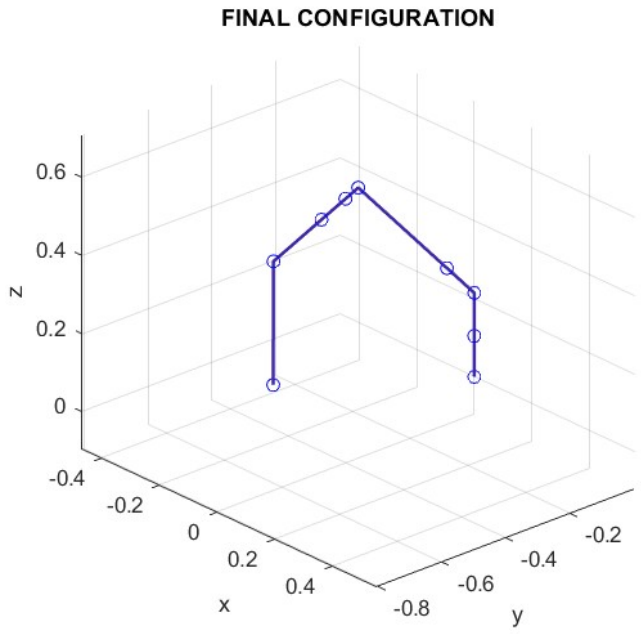


Figure 4: Final configuration of the manipulator.